11/09/24

Object Oriented Programming

What? (set of rules, ideas, concepts)
→ Programming paradigm based on the concept of "Objects" which are instances of classes.

→ Designed to model real-world problems by organizing code into "Objects" and "classes"
↳ entities with attributes (data) & behaviours (functions or methods)
→ User-defined datatypes that serves as a blue-print for creating objects.

→ Coding part under OOPs folder.

Note:
Members = Attributes + behaviours (methods)

Access Modifiers:

What?
→ Keywords that specify the accessibility (visibility / scope) of class members

Types?
① Public ② Private ③ Protected
↕ by default ↕
Struct Class

Constructor:

What?
→ A special type of method that is invoked each time a object of a class is created.

→ Compiler provides something called default constructor

Rules of creating our constructor:
① → No return type. (no void aswell)
② → Constructor will have the same name as the class.
③ → Needs to be public (but not all cases)

Note:
→ A class can have multiple constructors, a concept known as constructor overloading.

→ Each constructor must have different parameter list (either in number, type, or order of parameters).

→ Only constructors with the same parameter types and in the same order are not allowed as compiler might have a ambiguity.

NOTES:
→ Other programming paradigm exists.
→ No built-in support for OOP in C, but still we'll be able to implemet.
→ Python is a 'multi-paradigm' lang - meaning you can use it in various styles including procedural, OOP, functional

Procedural	Functional	Declarative
→ Sequential Execution Code is executed in a top-down, step-by-step manner, unless interrupted by function calls or control structures (subroutines) → C, Pascal, Basic etc..	→ Treats computations as the evaluation of mathematical functions. → Functions are "first-class citizens" where they can be assigned to variables, passed as arguments or returned as values → Haskell, Erlang	→ Focuses on what the program should accomplish, rather than how. → Describes logic without specifying control flow. → SQL, HTML

15/04/26

Key Pillars of OOPs

① Encapsulation:

What? → Bundling the data (attributes) and the methods (functions) that operate on the data into a single unit, called a class

→ Also involves restricting direct access to some of the object's components, which help protect the integrity of the data and hide internal implementation details from the outside interference.

→ Example for where encapsulation is needed → under OOPs folder.

Key Aspects?

① Data Hiding - By using access modifiers (private, protected, public), encapsulation hides the internal state of an object from outside access.

② Controlled Access - Provides controlled access to an object's internal state (private members) through a) getters and b) setters, which are public methods.

a) Getters: → Public method, read-only access to private data members.
→ Way to retrieve the value of a private variable without directly exposing the variable itself.

b) Setters: → Public method, allow controlled modification of private data members.
→ Often include validation logic to ensure only valid values are assigned to private variable.

③ Modularity: → Helps in organizing and modularizing code by bundling related data and functions together.

Note:
→ Constructor is not a getter/setter.
→ Employee* employee3 = new Employee();
↳ It dynamically allocates memory for an object of a variable on the heap (also known as dynamic memory) and returns a pointer to the allocated memory.
→ Heap vs Stack Memory
→ Interface - only abstract methods, Derived class - can have both abstract method and concrete method
→ Virtual functions and pure virtual functions, when over-ridden also needs to have the same parameters in the function header.

16/09/24

② Abstraction:

What?

Fundamental concept that involves hiding the complex implementation details of a system and exposing only essential features.

Types?

① Data Abstraction: → Hides the internal attributes or data of a class.

→ Allows you to interact with an object without needing to know the details of its data-structure.

② Process Abstraction: → Hides the internal implementation of methods.

→ Lets us use methods without understanding the underlying logic, or process.

16/09/24

How?

Abstraction in C++ is acheived through abstract classes and interfaces (via pure virtual functions).

18/09/24

~~Virtual Cl~~

Abstract Class:

What?

→ A class that cannot be instantiated directly and is intended to be a base class.

→ It contains atleast one pure virtual function.

Virtual Function:

What? → A member function in a base class that can be overridden in a derived class.

→ It allows dynamic (runtime) polymorphism.

→ Function in a base class one re-defined in derived class

Purpose: → Ensures that the correct function is called for an object, depending on the type of object pointed to, not the type of pointer.

Pure Virtual Function:

What? → It is a virtual function that has no implementation in the base class.

→ It must be overridden in any derived class.

→ A class with atleast one pure virtual function becomes an abstract class

Purpose: → It enforces derived classes to provide specific implementation for the function.

How are we able to acheive Abstraction from Abstract class?

→ Abstract class provides a high level interface that specifies what operations can be performed without specifying how these operations are implemented.

→ The actual details of implementation is left to derived classes.

→ Since the derived class must provide implementation, the abstract class is hidden from the implementation details of the specific method.

Notes:

→ An abstract class can contain virtual functions and normal (non-virtual) functions aswell.

→ Normal functions can have their own implementation in derived class, but do not provide the same dynamic polymorphism behaviour as overriding virtual functions. Calling the function depends on the type of the object (not the type of pointer or reference).

→ Derived Classes can have their own virtual functions and pure virtual functions, which can extend the abstraction in a multi-level manner.

19/04/24

Heap Vs Stack: (some details):

Stack Memory:

→ Automatically managed: Memory is freed when the function returns or the variable goes out of scope.

→ Used for:

a) Local variables (storing func parameters, return addresses, etc..)

b) Contiguous memory for variables.

c) Fast allocation and deallocation, but has limited size.

→ For vectors, vector objects (like metadata - size, capacity, pointer to the element) is on stack.

→ Examp: Employee emp("Ash"); (Employee: class name)

→ Referenced by '.' for class members.

Heap Memory:

→ Dynamically managed: You allocate (or malloc in) with 'new' and must manually free memory with 'delete' (or use smart pointers)

→ Used for:

a) Dynamic objects and large data-structures (eg, vector elements)

b) Objects that need to persist beyond the function scope.

→ Actual elements of the vector are stored contiguously on heap.

→ Non-contiguous memory allocation.

→ Flexible but requires memory management, or memory leaks can occur.

→ In Linked-list, Nodes are allocated dynamically on the heap.

→ Each node contains a pointer to the next, and nodes can be scattered in memory.

→ Vector elements are stored in heap because the size of the vector is dynamic and can change during run-time, which stack cannot provide.

→ RAM is typically divided into different regions for managing different types of memory allocation.

→ The OS and compiler allocate a portion of RAM for the Stack and portion to Heap.

(Stack: func calls, local variables, control flow)

→ Other types of memory are Global & Static Memory, Memory-Mapped I/O, Buffer Memory, Reserved Memory

③ Inheritance:

→ Fundamental concept in object-oriented programming (OOP) that allows one class (the derived class or child class or sub class) to inherit the properties and behaviours of another class (the base class or parent class or superclass).

→ This mechanism promotes code reusability and establishes a hierarchical relationship between classes

26/4/25

Types?

most common → ① public: Public & protected members of base stay the same in derived.

② protected: Public & protected become protected

default → ③ private: Public & protected become private in derived.

Constructor behaviour:

→ Base class constructor is called before the derived class

→ If base class has no default constructor, derived must explicitly call a parameterized base constructor

Access & Restrictions:

→ Private members of the base class are not directly accessible in the derived class

→ Use protected in base class if you want derived classes to access members but hide them from outside

Note:

→ If we create our own constructor in the base class, the default (no-arg) constructor is no longer provided automatically by the compiler.

27/4/25

④ Polymorphism:

What?

Fundamental concept in OOP. paradigm, that gives the ability for objects of different classes to use the same method (defined in superclass), but respond/behave differently/ in their own way (based on the class of the object that calls the method)

Types?

① Compile-time Polymorphism (Static Polymorphism):

→ A type of polymorphism that is resolved during compile time.

→ Implemented (in most cases) using function overloading & operator overloading

② Runtime Polymorphism (Dynamic Polymorphism):

→ A type of polymorphism that is resolved at run-time.

→ Typically involves method overriding.

Function over-loading:

→ A feature, where multiple functions can have same name, output type but different in no. of parameters or type of each parameter.

Operator over-loading:

→ A feature, where we can redefine the function of the operator i.e; we can change how it works

```
Sum = s1.add(s2);
Sum = s1 + s2
```

obj — like sum — other object

```
Student operator+ (Student other);
```

~~Runtime~~

Method Overriding:

→ A feature that involves a Sub-class providing a specific implementation of a method (same name, same parameters) that is already defined in its Super-class.

→ Virtual function in the base class helps in acheiving method over-riding.

→ Method in the base class is overridden by the method in Subclass.

27/4/25

④ Polymorphism:

What?

Fundamental concept in OOP. paradigm, that gives the ability for objects of different classes to use the same method (defined in superclass), but respond/behave differently/ in their own way (based on the class of the object that calls the method)

Types?

① Compile-time Polymorphism (Static Polymorphism):

→ A type of polymorphism that is resolved during compile time.

→ Implemented (in most cases) using function overloading & operator overloading

② Runtime Polymorphism (Dynamic Polymorphism):

→ A type of polymorphism that is resolved at run-time.

→ Typically involves method overriding.

Function over-loading:

→ A feature, where multiple functions can have same name, output type but differ in no. of parameters or type of each parameter.

Operator over-loading:

→ A feature, where we can redefine the functionality of the operator i.e; we can change how it works

```
Sum = s1.add(s2),
Sum = s2 + s2
```

obj — line sum — other object

```
Student operator+ (Student other){ }
```

~~Runtime~~

Method Overriding:

→ A feature that involves a sub-class providing a specific implementation of a method that is already defined in its super-class.

(same name: super-class, sub-class)

→ Virtual function in the base class helps in acheiving method over-riding.

→ Method in the base class is overridden by the method is Subclass.